Cloud Computing Technology

CSC/ECE 547 - Fall 2023

Online Bookstore Service

Ankur Banerji(abaner24), Vrushanki Patel(vpatel25)

North Carolina State University

Plagiarism Declaration

“We, the team members, understand that copying&pasting material from any source in our project is an allowed practice; we understand that not properly quoting the source constitutes plagiarism.

All team members attest that we have properly quoted the sources in every sentence/paragraph we have copied and pasted in our report. We further attest that we did not change words to make copy and past material appear as our work.”

Table of Contents

Table of Contents **2**

1 Introduction (5%) **4**
1.1 Motivation 4
1.2 Executive Summary 4

2 Problem Description (20%) **5**
2.1 The problem 5
2.2 Business Requirements 5
2.3 Technical Requirements 7
2.4 Tradeoffs and Conflicting Requirements 13

3 Provider Selection (20%) **15**
3.1 Criteria for choosing a provider 15
3.2 Provider Comparison 15
3.3 The final selection 17
3.4 The list of services offered by the winner 17

4 The first design draft (20%) **19**
4.1 Basic building blocks of the design 21
4.2 Top-level informal validation 22
4.3 Action items and rough timeline 22

5 Second design draft (20%) **25**
5.1 Use of the Well-Architected Framework 25
5.2 Discussion of pillars 27
5.3 Use of CloudFormation diagrams 29
5.4 Validation of the design 31
5.5 Design principles and best practices used 33
5.6 Tradeoffs revisited 34
5.7 Discussion of an alternate design 35

6 Kubernetes experimentation (15%) **36**
6.1 Experiment Design 36
6.2 Workload Generation with Locust 38
6.3 Analysis of the results 41

7 Ansible Playbooks (0%) **43**

8 Demonstration (0%) **44**

9 Comparisons (0%) **45**

10 Conclusion (5%) **46**
10.1 Lessons Learned 46
10.2 Possible continuation of the project 47
11 References **48**

1 Introduction (5%)

1.1 Motivation

In order to use the ideas we have learned in the cloud computing class, our group decided to choose a project topic that would allow us to do just that. Regarding the problem, we wanted to design a practical application that wouldn't require a lot of time to construct (as the lecturer has often said, "This is not a DevOps class").

1.2 Executive Summary

Online booksellers must overcome the difficulty of effortlessly delivering material to mobile customers in the current era of mobile accessibility. They are using cloud computing solutions, especially Content Delivery Networks (CDNs) from cloud service providers, to get around this. These CDNs improve mobile accessibility, lower latency, and optimize content delivery. This solution not only enhances the user experience but also supports the objective of effectively reaching a worldwide mobile audience. Cloud computing is completely changing how users and online booksellers interact, making books conveniently available on any mobile device.

2 Problem Description (20%)

2.1 Problem

An online bookshop service aims to solve the problem of offering a seamless user experience for mobile users in response to the growing significance of mobile accessibility and the growing demand for faster, more efficient content delivery. The issue at hand is how to best provide different kinds of content, including texts, photos, and videos, to mobile devices. The use of a cloud service provider's Content Delivery Network (CDN) is being explored as a means to do this.

In order to decrease latency and increase the speed at which content is delivered to mobile devices, this project aims to use a globally distributed CDN infrastructure to store and distribute content closer to end users. The online bookstore hopes to improve mobile accessibility by tackling this issue and making sure that mobile users can quickly and effectively access and engage with the material. This project not only enhances the user experience but also advances the overarching strategic goal of successfully reaching and interacting with a worldwide mobile audience.

Content Delivery Challenges:

Latency and Slow Load Times: Due to high latency, mobile users—who are typically on unreliable networks—frequently experience slow load times for photos and other material. This has a negative impact on the user experience and might cause users to become frustrated or stop using the site.

Dynamic Content: The platform's content is regularly updated and republished. To keep consumers interested and informed, it is imperative to guarantee the real-time transmission of fresh material and updates.

2.2 Business Requirements

1. *Scalability*: Ensure CDN can scale dynamically to handle increased mobile traffic as the user base grows.

2. *Cost Optimization*: Create optimal CDN costs while maintaining high performance, choosing pricing that fits the online bookstore’s budget.

3. *Global reach*: Use a CDN provider with a global network of servers to make content more efficiently available to users around the world.

4. *Content security*: Use CDN security features such as DDoS protection and HTTPS support to protect content and user data.

5. *Redundancy*: Redundancy Use CDN servers and data centers to ensure high availability and reliability.

6. *Data Analytics*: Use CDN analytics tools to gain insights into content delivery performance and user behavior, facilitating data-driven decision-making.

7. *Mobile Optimization*: Choose a CDN that optimizes content on mobile devices, improving the mobile user experience.

8. *Easy integration*: Choose a CDN with straightforward integration options and clear documentation to reduce implementation time and complexity.

9. *Real-time monitoring*: Establish real-time monitoring and alerts of CDN performance to quickly identify and address issues.

10. *Traffic Routing*: Use intelligent traffic routing to direct mobile users to the nearest CDN server, reducing latency.

11. *Load Balancing*: Use CDN load balancing to evenly distribute traffic across multiple servers to ensure optimal content delivery.

12. *Optimization*: Optimize the CDN system for specific mobile content delivery needs, such as adaptive streaming for video.

13. *Edge caching*: Use edge caching to serve frequently accessed objects from the cache, reducing server load and latency.

14. *HTTPS support*: Make sure the CDN supports secure connections (HTTPS) for mobile users, which provides reliability and security.

15. *Data Compression*: Implement real-time content compression to reduce file size and improve mobile content delivery speed.

16. *Content deletion*: Configure a method to purge expired or invalid content from the CDN cache in real time.

17. *Access control*: Implement access control in the CDN to restrict content access to authorized users and devices.

18. *Cost Management*: Constantly monitor CDN usage and costs to ensure effective and efficient cost management.

19. *Performance SLAs*: Develop service level agreements (SLAs) with the CDN provider to ensure that performance requirements are met at the performance level.

20. *Disaster Recovery*: Develop a disaster recovery plan that includes CDN failover strategies to maintain content availability during outages or disruptions.

2.3 Technical Requirements

1. Implement auto-scaling mechanisms to dynamically adjust the number of ECS instances in response to application demand. (1)

2. Configure CPU and memory usage thresholds to initiate scaling actions, ensuring resource availability aligns with traffic patterns. (1)

3. TR: Utilize cost management tools to monitor and control expenditure on CDN and other AWS services. (2)

4. TR: Deploy CDN with edge locations in multiple geographic regions to decrease content delivery times to users worldwide. (3)

5. TR: Employ CDN security features such as SSL/TLS encryption for data in transit. (4)

6. TR: Configure DDoS protection measures and web application firewalls to safeguard against external threats. (4)

7. TR: Design the system for multi-AZ deployment to ensure high availability and fault tolerance. (5)

8. TR: Implement failover strategies to switch traffic to backup resources in the event of a component failure. (5)

9. TR: Integrate CDN and application logs with analytics tools for real-time performance monitoring. (6)

10. TR: Utilize data visualization tools to interpret user engagement and content delivery efficiency. (6)

11. TR: Enable mobile-specific content adaptations in the CDN configuration to optimize delivery to mobile devices. (7)

12. TR: Ensure API and SDK availability for seamless integration of CDN services with existing application infrastructure. (8)

13. TR: Set up cloud monitoring services to provide real-time alerts and dashboards for system performance metrics. (9)

14. TR: Configure monitoring tools to track and alert on key performance indicators (KPIs) and system health. (9)

15. TR: Implement intelligent DNS resolution services to route user requests to the optimal CDN endpoint. (10)

16. TR: Configure load balancers to distribute incoming traffic evenly across ECS service instances. (11)

17. TR: Set up health checks to ensure traffic is only routed to healthy instances. (11)

18. TR: Leverage adaptive streaming technologies to deliver video content efficiently across various network conditions. (12)

19. TR: Optimize CDN configurations for the specific content types predominant in the online bookstore, such as e-books and images. (12)

20. TR: Configure edge caching mechanisms within the CDN to store and serve frequently requested content, reducing latency and backend load. (13)

21. TR: Ensure all components of the application enforce HTTPS to protect data integrity and confidentiality. (14)

22. TR: Manage SSL/TLS certificates and renewal processes through a centralized certificate management service. (14)

23. TR: Apply real-time compression algorithms for static and dynamic content to reduce payload sizes and improve content delivery speeds. (15)

24. TR: Utilize content delivery network features to serve compressed content based on client capabilities. (15)

25. TR: Implement automated purging systems within the CDN to remove outdated or invalid content from caches. (16)

26. TR: Implement fine-grained access control policies to restrict content access to authorized users and services. (17)

27. Implement tenant identification mechanisms to distinguish and manage access for different tenants within the application. (17)

28. TR: Monitor and optimize CDN and cloud resource usage to maintain operational efficiency within budget constraints. (18)

29. TR: Develop and enforce service level agreements with cloud service providers to guarantee the required performance levels. (19)

30. TR: Set up performance monitoring to ensure compliance with SLAs and to trigger adjustments as needed. (19)

31. TR: Create a disaster recovery plan that includes strategies for data backup, system failover, and quick restoration of services. (20)

32. TR: Regularly test disaster recovery procedures to ensure system resilience and data integrity. (20)

BR(s)	**Associated TR(s)**	**Justification**
BR 1	TR 1, TR 2	● Ensure the application can handle the

		increased load without manual intervention, which is critical for user experience and system reliability. (TR 1) ● Allow the system to maintain optimal performance by automatically scaling resources based on actual demand, improving resource efficiency. (TR 2)
BR 2	TR 3	● Help in managing costs by monitoring the use of resources, which is essential for maintaining a budget. (TR 3)
BR 3	TR 4	● Reduce latency for a global audience by bringing content closer to users, which is essential for a fast user experience. (TR 4)
BR 4	TR 5, TR 6	● Protect data during transit, which is essential for maintaining trust and complying with data protection regulations. (TR 5) ● Protect the application against common internet threats, ensuring uptime and reliability. (TR 6)
BR 5	TR 7, TR 8	● Ensure that the application remains available even if one AZ goes down, which is crucial for maintaining service continuity. (TR 7) ● Enable the system to quickly recover from failures by rerouting traffic, minimizing downtime. (TR 8)
BR 6	TR 9, TR 10	● Provide insights into application performance and user behavior, which is crucial for continuous improvement. (TR 9) ● Help in making informed decisions by visualizing data trends and patterns. (TR 10)
BR 7	TR 11	● Enhance the experience for mobile users, who form a large part of internet traffic, by optimizing content for mobile delivery. (TR 11)

BR 8	TR 12	● Facilitate the expansion and maintenance of services by allowing easy integration with other systems. (TR 12)
BR 9	TR 13, TR 14	● Allow for immediate response to performance issues, which is critical for maintaining a high-quality user experience. (TR 13) ● Ensure that performance remains within acceptable thresholds, which is essential for service reliability. (TR 14)
BR 10	TR 15	● Ensure optimal response times by intelligently routing requests, which is essential for user satisfaction. (TR 15)
BR 11	TR 16, TR 17	● Ensure no single instance is overloaded, which improves the application's responsiveness and stability. (TR 16) ● Prevent traffic from being sent to failed instances, improving the overall reliability of the service. (TR 17)
BR 12	TR 18, TR 19	● Ensure efficient delivery of video content, which can be bandwidth-intensive, improving user experience. (TR 18) ● Tailor the CDN for the specific needs of a bookstore, enhancing the delivery of relevant content types. (TR 19)
BR 13	TR 20	● Decrease load times by storing content closer to users, reducing the load on backend servers. (TR 20)
BR 14	TR 21, TR 22	● Protect user data and maintain data integrity, which is a basic requirement for modern web applications. (TR 21) ● Simplify certificate management, ensuring ongoing security compliance without manual overhead. (TR 22)
BR 15	TR 23, TR 24	● Improve load times by reducing the size of the content being delivered, which is particularly important for users

		with limited bandwidth. (TR 23) ● Ensure the most efficient use of network resources by delivering content in formats suited to the user's device. (TR 24)
BR 16	TR 25	● Help in managing content freshness by removing outdated items, which is important for maintaining user trust and reducing storage costs. (TR 25)
BR 17	TR 26, TR 27	● Restrict access to sensitive content, which is necessary for protecting intellectual property and user privacy. (TR 26) ● Maintain data privacy and ensure that each tenant's information and interactions are securely segregated, which is a cornerstone of multi-tenant system security. (TR 27)
BR 18	TR 28	● Tracking resource utilization against the budget, ensuring that the application remains cost-effective. (TR 28)
BR 19	TR 29, TR 30	● Guarantee that cloud services meet agreed performance standards, which is essential for customer satisfaction and business reputation. (TR 29) ● Ensure ongoing adherence to performance benchmarks, allowing for proactive adjustments to maintain service levels. (TR 30)
BR 20	TR 31, TR 32	● Ensuring that in the event of a disaster, the impact on services and data is minimal. (TR 31) ● Ensuring that the system can recover swiftly and with integrity when needed. (TR 32)

2.4 Tradeoffs

1. [TR1 conflicts with TR6]:
   Auto-scaling for elasticity (TR1) could conflict with cost optimization (TR6). Scaling up instances improves performance but may increase costs.

2. [TR4 conflicts with TR18]:
3. Deploying CDN globally (TR4) for reach might conflict with optimizing for mobile-specific content (TR18), as regional content adaptation requires more complex configurations.

4. [TR7 conflicts with TR20]:
   Multi-AZ deployment for redundancy (TR7) could increase latency due to data replication, conflicting with edge caching (TR20) aimed at reducing latency.

5. [TR11 conflicts with TR28]:
   Load balancing to distribute traffic (TR11) could lead to higher costs, conflicting with the need to monitor and optimize resource usage for cost management (TR28).

6. [TR13 conflicts with TR30]:
   Real-time monitoring (TR13) demands resources, potentially conflicting with performance SLAs (TR30) if the monitoring overhead impacts overall system performance.

7. [TR8 conflicts with TR19]:
   Implementing failover strategies (TR8) can increase complexity and may conflict with optimizing CDN configurations (TR19) for specific content types, as each failover scenario might require custom CDN settings.

8. [TR9 conflicts with TR26]:
   Real-time analytics (TR9) for user behavior can conflict with access control measures (TR26), where the need for detailed data could potentially clash with privacy and data access policies.

9. [TR12 conflicts with TR27]:
   Seamless integration of CDN services (TR12) might conflict with tenant identification (TR27) due to varying integration requirements that could affect multi-tenancy logic and data isolation.

10. [TR16 conflicts with TR22]:
    Configuring load balancers (TR16) for optimal traffic distribution may conflict with managing SSL/TLS certificates (TR22), especially when certificates need to be deployed across multiple load balancers or regions.

11. [TR5 conflicts with TR24]:
    Applying security measures like SSL/TLS (TR5) can increase latency, which may conflict with the goal of real-time content compression (TR24) designed to speed up content delivery.

12. [TR3 conflicts with TR31]:
    Utilizing cost management tools (TR3) to control expenditure can conflict with disaster recovery plans (TR31) that require additional resources, such as backup storage, which may lead to increased costs.

13. [TR10 conflicts with TR21]:
    Using data visualization tools (TR10) for interpreting user engagement may require complex data processing, which could conflict with enforcing HTTPS (TR21) if the tools do not support encrypted connections.

3 Provider Selection (20%)

3.1 Provider Selection Criteria:

1. *Service Availability*: The chosen cloud provider must offer a comprehensive service catalog, encompassing all the necessary services to fulfill our Technical Requirements (TRs).

2. *Industry Standing*: It is crucial that the cloud provider maintains a positive reputation within the IT industry. It should not be notorious for frequent outages, data breaches, or other detrimental incidents.

3. *Team Comfort*: All team members should feel at ease working with the selected cloud provider. Familiarity with the platform ensures efficient collaboration and smoother implementation of Technical Requirements.

4. *Cost Competitiveness*: Optimal pricing is essential. The selected cloud provider should offer the most competitive and cost-effective pricing models for the desired services.

5. *Locations*: A key criterion is the global reach of the cloud provider. It should have a widespread presence with data centers in various locations, ensuring reliable and low-latency services worldwide.

3.2 Provider Comparison:

Provider	**AWS**		**GCP**		**Azure**		**Cloudfare**	
Parameter	**Justification**	**Score**	**Justification**	**Score**	**Justification**	**Score**	**Justification**	**Score**
Service Availability	Has the largest service catalog (200+)	2	Has the smallest service catalog out of the big 3	0	Has the second biggest catalog	1	extensive service catalog with over 200 services	1
Industry Standing	has the largest	2	the smallest of the Big 3	2	Second only to AWS in terms of	2	Cloudflare, is trusted by a diverse	2

	market share of the three major providers, serving companies like SAP, Netflix, and Expedia.		but with clients like Spotify and Twitter		market share; preferred by companies using Microsoft tools		range of clients, including notable entities such as Reddit, Mozilla, and OkCupid	
Team Comfort	Highest familiarity and experience	2	fundamental expertise and understandin g of services	1	No familiarity or experience	0	No familiarity or experience	0
Cost Competitiven ess	AWS gives customers who commit for a year a 41% savings. Even yet, AWS ends up being less expensive than the other two.	2	GCP offers a 63% discount for a 1 year commitment.	1	Azure offers a 41% discount for a 1 year commitment.	1	No discounts	0
Locations	30 regions and 96 availability zones across all continents	1	35 regions and 106 zones across all continents except Africa	0	60+ regions with multiple availability zones per region across all continents	2	120+ countries availibilty	1

3.3 Final Selection:

In general, the services provided by the Big Four are comparable. But we will choose AWS over the other cloud providers because of their appealing pricing structures, extensive service offering, and—most importantly—our familiarity with AWS. As a result, it also contributes to the highest overall score.

3.4 Services offered by winner:

a) **Elastic Compute Cloud (EC2)**: Several compute instance types are available for a range of purposes; visit https://aws.amazon.com/ec2/ . With its autoscaling features, it makes your program elastic and supports the newest Intel, AMD, and Arm processors. It also provides cost savings through per-second charging and CPU optimization.

b) **Amazon S3** - https://aws.amazon.com/s3/

Block storage is offered by Amazon's Simple Storage Service, or S3. It can hold up to 5TB worth of "objects," which include text files, videos, pictures, and other types of media. Buckets are used to hold these items. Along with a slew of other capabilities, AWS guarantees 11 9s of durability for S3. These features include:

1. *Security*: Bucket rules and access control lists can be used to restrict access to buckets and the items within them.
2. *Replication*: For increased redundancy, S3 items can be duplicated throughout AWS regions via S3 replication.
3. *Storage classes*: For a certain monthly monitoring fee, AWS may automatically divide your data into tiers (frequent, infrequent, archive, and deep-archive) based on how frequently it is accessed.

c) **DynamoDB:** https://aws.amazon.com/dynamodb/

The managed key-value database service has latency ranging from milliseconds to microseconds. Global replication, auto-scaling, encryption at rest, and interaction with other AWS services are some of the features that DynamoDB offers.

d) **Elastic Load Balancing**: https://aws.amazon.com/elasticloadbalancing/

Traffic is dispersed among a range of target IP addresses. Three variations of ELB are available: gateway load balancing, network load balancing, and application load balancing.

e) **Elastic Kubernetes Service:** https://aws.amazon.com/eks/
It's a Kubernetes managed service. The program may be made extremely scalable by using containers running in Fargate or AWS EC2.

f) **CloudWatch** - https://aws.amazon.com/cloudwatch/
A logging and monitoring service is called CloudWatch. Logs from various services can be stored, accessed, and analyzed by users through the AWS catalog.

g) **Identity and Access Management (IAM)**: https://aws.amazon.com/iam/
Using IAM, a company can effectively and centrally specify what or who is able to access which services.

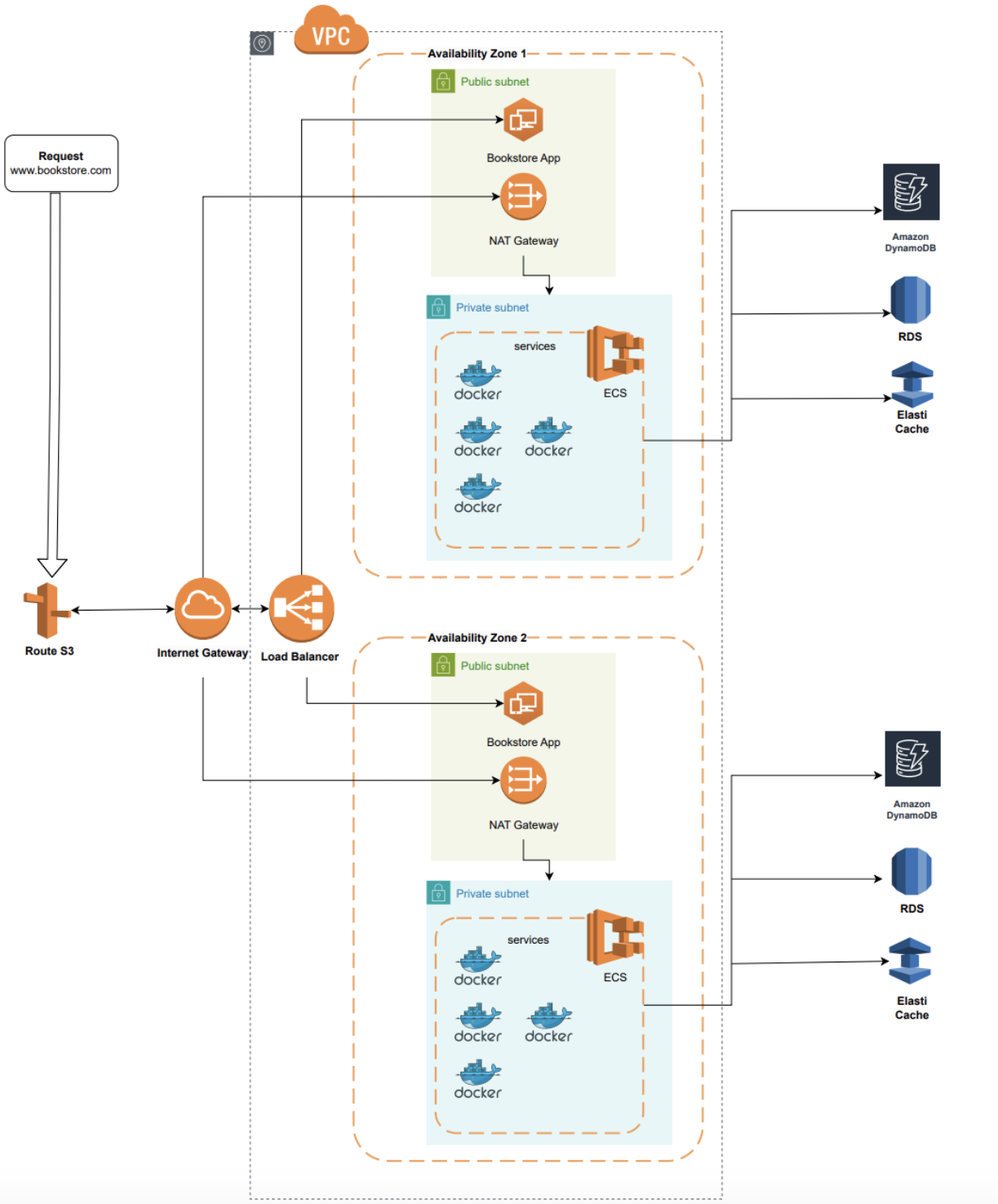
h) **Virtual Private Cloud**: https://aws.amazon.com/vpc/
It's a service that enables you to set up a virtual environment that makes sense. You may manage internal IP addressing, load balance, control incoming and outbound traffic, isolate different parts of your application, and improve security using security groups by using virtual private clouds (VPCs).

4 The first design draft (40%)

We select the following TRs from the list supplied in section 2.4 →

1. Implement auto-scaling mechanisms for ECS instances. (TR 1: Scalability)
2. Configure thresholds for scaling based on CPU and memory usage. (TR 2: Scalability)
3. Multi-AZ deployment for high availability and fault tolerance. (TR 7: Redundancy)
4. Failover strategies for component failure. (TR 8: Redundancy)
5. Cloud monitoring services for real-time system performance. (TR 13: Real-time Monitoring)(Performance Efficiency)
6. Monitoring tools for KPIs and system health. (TR 14: Real-time Monitoring)
7. Load balancers to distribute incoming traffic. (TR 16: Load Balancing)
8. Health checks for routing traffic to healthy instances. (TR 17: Load Balancing)
9. Enforce HTTPS for data integrity and confidentiality. (TR 21: HTTPS Support)(Security)
10. Centralized management of SSL/TLS certificates. (TR 22: HTTPS Support)
11. Fine-grained access control policies. (TR 26: Access Control)
12. Implement tenant identification mechanisms to distinguish and manage access for different tenants within the application. (TR 27: Access Control)(Tenant Identification)
13. Disaster recovery plan for data backup and system restoration. (TR 31: Disaster Recovery)(Reliability)
14. Regular testing of disaster recovery procedures. (TR 32: Disaster Recovery)

A preliminary version of the architecture is shown below:


VPC
Availability Zone 1
Public subnet
Bookstore App
NAT Gateway
Private subnet
services
ECS
docker
docker
docker
docker
Request
www.bookstore.com
Route S3
Internet Gateway
Load Balancer
Availability Zone 2
Public subnet
Bookstore App
NAT Gateway
Private subnet
services
ECS
docker
docker
docker
docker
Amazon DynamoDB
RDS
Elasti Cache
Amazon DynamoDB
RDS
Elasti Cache

4.1 The basic building blocks of the design

1. Deployment and Management Automation
    a. Infrastructure as Code (IaC) Services: Automates provisioning and management of infrastructure, ensuring environment consistency.
    b. Continuous Integration and Continuous Deployment (CI/CD) Pipelines: Streamlines code updates, facilitating rapid and reliable application changes.
    c. Elastic Container Registry (ECR): Provides a managed Docker container registry, simplifying storage, management, and deployment of container images.

2. Data Storage Solutions
    a. S3 Buckets: Serve as repositories for static content, including media files and document assets.
    b. Relational Database Service (RDS): Hosts the application's transactional data in a structured format for robust query capabilities.
    c. DynamoDB: Provides a quick-access NoSQL database platform for user profiles and operational logging.

3. Application Processing Infrastructure
    a. Container Service (ECS): Manages containerized application components, streamlining deployment and scaling.
    b. Elastic Compute Cloud (EC2): Powers the application's back-end processing and containerized tasks.

4. Connectivity and Network Framework
    a. DNS Management (Route 53): Translates the application's domain names into IP addresses for browser access.
    b. Virtual Private Cloud (VPC): Structures a secure, isolated network space, complete with public and private subnet segmentation.
    c. Internet and NAT Gateways: Facilitate outbound internet communication for services while maintaining inbound access control.

5. Operational Monitoring and Access Security
    a. Identity Access Management (IAM): Enforces user permissions and secure access protocols.
    b. Cloud Monitoring (CloudWatch): Tracks system operations, audits performance metrics, and oversees logging data.

4.2 Top-level, informal validation of the design

1. TR1 & TR2 - ECS Auto-Scaling: Connected to the Application Processing Infrastructure's ECS, which manages containerized components and scales with demand, using CloudWatch from Operational Monitoring for metrics-based scaling decisions.

2. TR21 & TR22 - CDN Security and HTTPS: Linked to Amazon CloudFront for CDN and AWS Certificate Manager for managing SSL/TLS certificates, part of the Operational Monitoring and Access Security.

3. TR7 & TR8 - Multi-AZ Deployment and Failover: Related to the VPC's ability to structure a secure network across multiple AZs, providing high availability and fault tolerance.

4. TR13 & TR14 - Real-time Monitoring and Alerts: Utilizes AWS CloudWatch from Operational Monitoring to provide real-time system performance metrics and alerts.

5. TR16 & TR17 - Load Balancing and Health Checks: Associated with AWS Elastic Load Balancing in the Connectivity and Network Framework to distribute traffic and ensure health checks.

6. TR26 & TR27 - Access Control and Tenant Identification: Connected to Amazon Cognito and IAM for managing user authentication and access control.

7. TR31 & TR32 - Disaster Recovery Planning: Relates to AWS backup services and the multi-AZ strategy of the VPC for creating and testing disaster recovery plans.

4.3 Action items and rough timeline

1. Project Initialization
    - Establish project management office (PMO) and define project scope.
    - Duration: 1 week
2. Resource Allocation

- Procure necessary AWS services subscriptions.
- Assemble project team and assign roles.
- Duration: 1-2 weeks

3. Design Phase
    - Finalize system architecture design.
    - Create a detailed design document for each system component.
    - Duration: 3 weeks
4. Development Phase
    - Setup of core AWS infrastructure (VPC, Subnets, IGW, etc.).
      Duration: 2 weeks
    - Development of application components in ECS with Docker containers.
      Duration: 4 weeks
    - Implementation of S3 for static asset storage.
      Duration: 1 week
    - Setup and configuration of DynamoDB tables.
      Duration: 2 weeks
    - Establishment of backup and snapshot procedures.
      Duration: 1 week
5. Security Implementation
    - Configuration of IAM roles and policies.
    - Encryption setup for data at rest and in transit.
    - Duration: 2 weeks
6. Performance Tuning
    - Setup auto-scaling for ECS and EC2 instances.
    - Fine-tuning of Elastic Load Balancer settings.
    - Duration: 2 weeks
7. Testing Phase
    - Integration testing of components.
    - Performance testing and load balancing optimization.
    - Security testing and vulnerability assessments.
    - Duration: 4 weeks
8. Monitoring Setup
    - Configuration of CloudWatch for system monitoring.
    - Setup of alarms and notifications.
    - Duration: 1 week
9. Documentation and Compliance
    - Development of system operation manuals.
    - Compliance checks and documentation.
    - Duration: 2 weeks
10. Deployment Phase

- Initial deployment to staging environment.
  Duration: 1 week
- Testing and bug fixes in staging.
  Duration: 2 weeks
- Production deployment.
  Duration: 1 week

11. Post-Deployment
    - Monitoring system performance in production.
    - User feedback collection and initial support.
    - Duration: Ongoing with the first review at 2 weeks post-deployment
12. Project Review and Closure
    - Final project review against objectives.
    - Documentation of lessons learned.
    - Formal project closure and handover.
    - Duration: 1 week

Total Estimated Project Duration: 6 months.

5 The second design (20%)

5.1 Use of Well-Architected Framework

1. TR1 & TR2 (Auto-scaling and Resource Thresholds): These are directly in line with the Performance Efficiency pillar, ensuring the system can maintain stable performance under varying loads. Auto-scaling also touches on Cost Optimization as it prevents over-provisioning.

2. TR3 (Cost Management Tools): This falls under Cost Optimization, enabling ongoing cost management and ensuring that expenditure on AWS services is tracked and optimized.

3. TR4 (CDN with Global Edge Locations): This supports the Performance Efficiency pillar by bringing content closer to users, thereby reducing latency, which also impacts the Reliability pillar by distributing content across a global network to handle regional demand effectively.

4. TR5 & TR6 (CDN Security Features and DDoS Protection): These align with the Security pillar, ensuring that the system encrypts data in transit and is protected against common cyber threats.

5. TR7 & TR8 (Multi-AZ Deployment and Failover Strategies): These technical requirements are key to the Reliability pillar, ensuring the system remains operational even in the event of an AZ failure.

6. TR9 & TR10 (Analytics Integration and Data Visualization): The ability to analyze and visualize data is crucial for Operational Excellence, allowing for informed decision-making and continuous improvement.

7. TR11 (Mobile-specific Content Adaptation): This is part of the Performance Efficiency pillar, ensuring that content is delivered optimally to mobile devices, which is increasingly important in user experience.

8. TR12 (API and SDK Availability for Integration): This aligns with Operational Excellence by ensuring that integration is as frictionless as possible, reducing time to market for new features.

9. TR13 & TR14 (Cloud Monitoring and Alerts): Continuous monitoring is a practice recommended by the Operational Excellence and Security pillars to ensure system health and security posture.

10. TR15 (Intelligent DNS Services): Routing with services like Amazon Route 53 supports the Performance Efficiency and Reliability pillars by reducing latency and managing traffic across the globe efficiently.

11. TR16 & TR17 (Load Balancing and Health Checks): AWS Elastic Load Balancing ensures traffic distribution and system health, which is crucial for both Performance Efficiency and Reliability.

12. TR18 & TR19 (Adaptive Streaming and CDN Optimization): These practices enhance user experience and system performance, aligning with the Performance Efficiency pillar.

13. TR20 (Edge Caching): Caching content at edge locations to reduce latency and load on origin servers is a recommendation under the Performance Efficiency pillar.

14. TR21 & TR22 (HTTPS Enforcement and SSL/TLS Management): These are key components of the Security pillar, essential for protecting data integrity and confidentiality.

15. TR23 & TR24 (Data Compression and Content Delivery): Compressing data for faster delivery aligns with Performance Efficiency and is a practice that can also impact Cost Optimization positively.

16. TR25 (Automated Content Purging): This is a practice related to both Operational Excellence for system efficiency and Security to ensure outdated content is not served.

17. TR26 & TR27 (Access Control and Tenant Identification): Fine-grained access controls are part of the Security pillar, necessary for data protection and compliance.

18. TR28 (Resource Usage Optimization): Monitoring CDN and cloud resource usage and optimizing it is essential for Cost Optimization, ensuring the system does not incur unnecessary expenses.

19. TR29 & TR30 (SLAs and Performance Monitoring): Developing and enforcing SLAs and monitoring performance align with the Reliability and Operational Excellence pillars.

20. TR31 & TR32 (Disaster Recovery and Testing): A comprehensive disaster recovery plan and regular testing of these strategies are crucial for Reliability, ensuring the system's resilience.

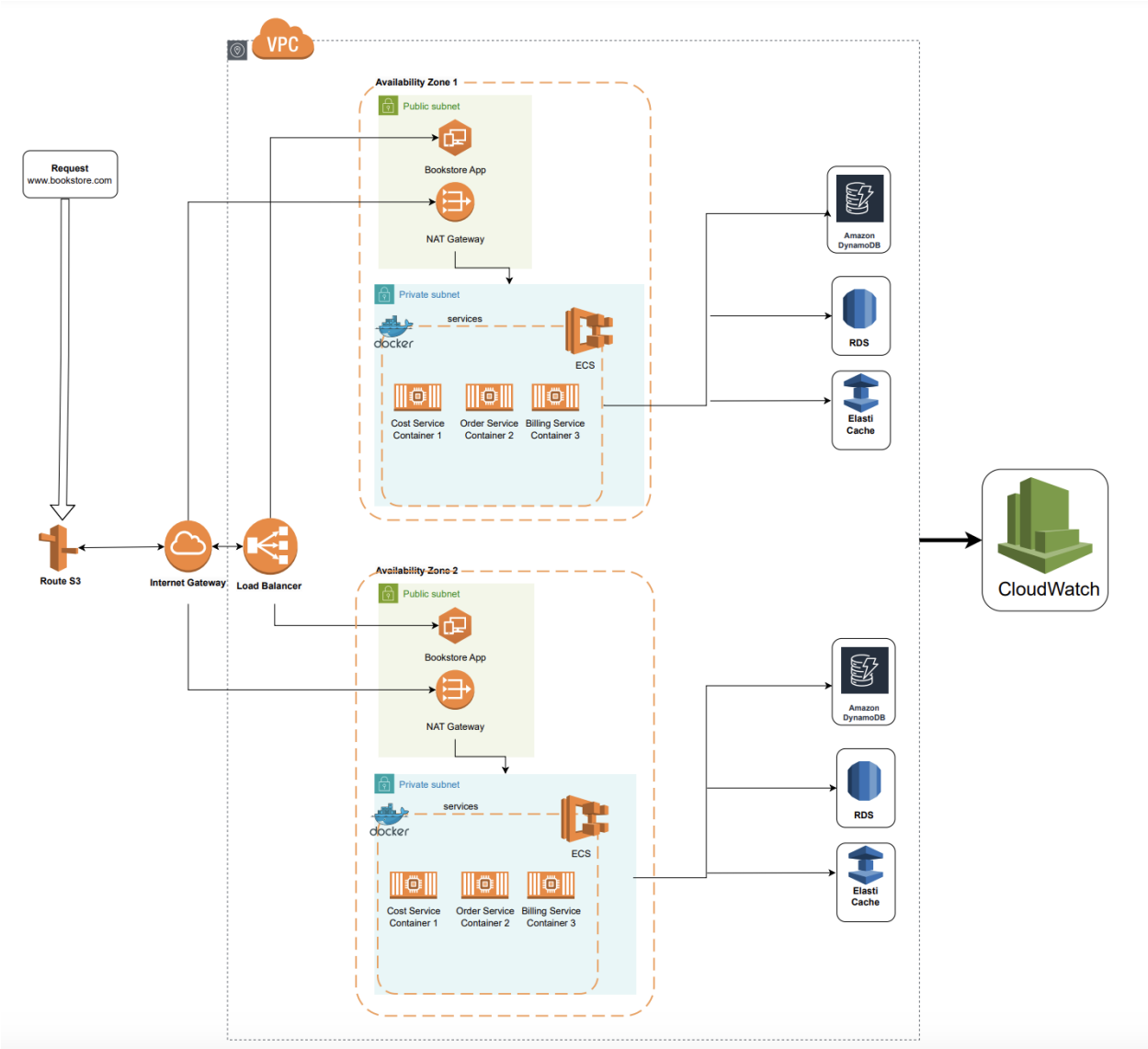
5.2 Discussion of Pillars

The AWS Well-Architected Framework states several pillars to build robust, secure, and scalable cloud infrastructure solutions. In particular, we'll be discussing two pillars - Security and Cost.

1. SECURITY:
    - Implement a strong identity foundation:
      This involves using AWS Identity and Access Management (IAM) to assign appropriate permissions to users and systems, ensuring that only authorized entities can access certain AWS resources.
    - Enable traceability:
      Monitor, alert, and audit actions and changes to environments in real-time using services like AWS CloudTrail and AWS Config.
    - Apply security at all layers:
      Rather than focusing security on the perimeter, apply it at the load balancer, instance, operating system, and application layers.
    - Automate security best practices:
      Automate security tasks to reduce the risk of human error. For example, use AWS Lambda to automate the response to security incidents.
    - Protect data in transit and at rest:
      Use encryption, tokenization, and access control mechanisms to safeguard your data.
    - Keep people away from data:
      Use mechanisms to reduce or eliminate the need for direct access or manual processing of data.
    - Prepare for security events:
      Have an incident management process that includes escalation paths, stakeholder engagement, and postmortem analysis

2. COST:
    - Implement cloud financial management: Understand and control where money is being spent by using AWS Budgets and AWS Cost Explorer.
    - Adopt a consumption model: Pay only for computing resources you need and increase or decrease usage depending on business requirements without upfront investment.
    - Measure overall efficiency: Use CloudWatch to track the efficiency of your resources and workloads to make informed decisions about resource allocation.
    - Stop spending money on undifferentiated heavy lifting: Avoid investing time and resources on IT infrastructure maintenance that doesn't differentiate your business.
    - Analyze and attribute expenditure: Use cost allocation tags and AWS Cost and Usage Reports to understand and track your AWS spending across different teams or projects.

5.3 Use of Cloudformation Diagrams



1. Route 53:
   AWS Route 53 is a scalable Domain Name System (DNS) web service. It is crucial for the online bookstore, as it provides a reliable and cost-effective way to route end users to Internet applications by translating names like 'www.bookstore.com' into the numeric IP addresses like 192.0.2.1 that computers use to connect to each other. Route 53 is effectively used to route mobile users to the nearest CDN endpoint, reducing latency, which is a key aspect of improving the mobile user experience.

2. Internet Gateway:
   This component in the architecture allows the network within AWS to communicate with the outside world over the internet. It is a pathway for data to travel in and out of the cloud, which is essential for a globally distributed system meant to serve a worldwide audience, as the online bookstore aims to do.

3. Load Balancer:
   This distributes incoming application traffic across multiple instances, such as EC2 instances, Docker containers, or Lambda functions. By doing so, it enhances the system's reliability and availability. For mobile users, this ensures that the system can handle the influx of traffic without degradation in performance, contributing to a seamless user experience.

4. Elastic Container Service (ECS):
   ECS manages Docker containers on a cluster. It's used to host the bookstore's microservices such as the Cost Service, Order Service, and Billing Service. This service is critical for the application's scalability and flexibility in resource usage, ensuring that the bookstore can handle the variable load from mobile users efficiently.

5. RDS/ElastiCache:
   These services are used to manage relational and in-memory data respectively. RDS supports complex transaction processing, while ElastiCache helps to improve the speed of data retrieval, which is vital for providing quick responses to mobile users' requests.

6. CloudWatch:
   This monitoring service provides real-time insights into application performance, operational health, and resource utilization. It enables the online bookstore to monitor their CDN performance and user engagement, ensuring that the content is being delivered efficiently to mobile users.

7. DynamoDB:
   As a NoSQL database service, DynamoDB offers fast and predictable performance with seamless scalability. For the bookstore, this means being able to store and retrieve any amount of data, serving any level of request traffic efficiently. It's especially useful for mobile users who expect quick interactions with the application.

8. Public and Private Subnets:

These are used to control access to the bookstore application and its microservices. The public subnet hosts the Bookstore App, making it accessible from the internet, while the private subnet secures backend services and databases that should not be directly exposed to the public internet.

5.4 Validation of the design

1. Scalability (BR1/TR1/TR2): The design includes Amazon ECS managed within ASGs, facilitating horizontal scalability in line with BR1. The auto-scaling mechanism (TR1) automatically adjusts ECS instances, scaling up with increased traffic, indicated by real-time demand metrics [1] [2]. The system also employs metric-based scaling (TR2), utilizing CloudWatch to monitor and trigger scaling based on predefined CPU and memory usage thresholds, ensuring the application adapts to user load. [15]

2. Global Reach (BR3/TR4): To achieve BR3, the design deploys a CDN (TR4) with a global presence, using Amazon CloudFront to cache content at edge locations worldwide. This strategy decreases content delivery times and improves user experience across different regions, enhancing the bookstore's global accessibility. [4] [5]

3. Content Security (BR4/TR5/TR6): The system's content security is fortified by implementing SSL/TLS encryption (TR5) for data in transit, using AWS Certificate Manager for managing certificates. Additionally, AWS Shield and AWS WAF (TR6) provide DDoS protection and application-level firewall services, contributing to a comprehensive security stance that meets BR4. [5] [6]

4. Redundancy (BR5/TR7/TR8): Multi-AZ deployment ensures redundancy and high availability, addressing BR5. If one AZ fails, the failover mechanism (TR7) redirects traffic to another AZ, minimizing downtime. Additionally, ASGs (TR8) across multiple AZs provide failover capacity, ensuring consistent application performance and reliability. [7] [13]

5. Data Analytics (BR6/TR9/TR10): Data analytics (TR9) is conducted through integration with AWS analytics services, allowing real-time performance monitoring of CDN and user interactions. The use of data visualization tools (TR10) helps in interpreting complex data sets, providing actionable insights that align with BR6. [9] [15]

6. Mobile Optimization (BR7/TR11): The CDN configuration is optimized (TR11) to serve mobile users more effectively, ensuring content is delivered in a format suitable for mobile devices, thus enhancing the mobile user experience and meeting BR7. [11] [5]

7. Easy Integration (BR8/TR12): The API Gateway (TR12) provides straightforward integration options for third-party services and internal microservices, reducing the time and complexity involved in implementing the CDN, supporting BR8. [12] [13]

8. Real-time Monitoring (BR9/TR13/TR14): The CloudWatch service (TR13) offers real-time monitoring and alerting capabilities for system metrics, ensuring any performance degradation is quickly identified and addressed, satisfying BR9. KPI tracking (TR14) aids in maintaining system health. [15]

9. Traffic Routing (BR10/TR15): Amazon Route 53 (TR15) is employed for intelligent DNS resolution, routing requests efficiently to the nearest CDN endpoint, improving load times, and fulfilling BR10. [3]

10. Load Balancing (BR11/TR16/TR17): AWS Elastic Load Balancing (TR16) evenly distributes incoming traffic across ECS instances. Health checks (TR17) are in place to ensure that traffic is only sent to operational instances, optimizing content delivery and achieving BR11. [13]

11. Optimization (BR12/TR18/TR19): The bookstore leverages adaptive streaming (TR18) for efficient video delivery, and optimizes the CDN (TR19) specifically for the bookstore's content, like e-books and images, to meet BR12. [5]

12. Edge Caching (BR13/TR20): Edge caching (TR20) is implemented to serve frequently requested content from the cache, significantly reducing latency and load on the origin server, thus addressing BR13. [14]

13. HTTPS Support (BR14/TR21/TR22): The design mandates HTTPS (TR21) across all components of the application, managed through a centralized service, ensuring data integrity and confidentiality. SSL/TLS certificate management (TR22) is automated for renewal and deployment processes. [14] [10]

14. Data Compression (BR15/TR23/TR24): Real-time data compression (TR23) is applied to reduce file sizes, which is crucial for mobile content delivery, while the

CDN is utilized to serve compressed content (TR24) efficiently, in line with client capabilities, meeting BR15. [5]

15. Access Control (BR17/TR26/TR27): Access control policies (TR26) restrict content access to authorized users, while tenant identification mechanisms (TR27) manage differentiated [10].

5.5 Design principles and best practices used

1. Elasticity and Scalability (TR 1 & TR 2): We applied the principle of elasticity to ensure our architecture can automatically scale resources in response to traffic fluctuations. This is achieved through auto-scaling ECS instances and configuring scaling thresholds based on CPU and memory usage.

2. High Availability (TR 7): Our design embraces the principle of high availability by leveraging multi-AZ deployments, ensuring that the application remains operational and fault-tolerant even during partial cloud service outages.

3. Operational Excellence (TR 13 & TR 14): By incorporating cloud monitoring services and tools to oversee KPIs and system health, we strive for operational excellence, allowing for real-time insights and proactive management of system performance.

4. Security (TR 21 & TR 26): Data integrity and confidentiality, enforced through HTTPS and fine-grained access control, reflect the security best practices, ensuring data protection across all system interactions.

Tradeoffs between TRs:

While striving for high availability and fault tolerance, we had to consider the cost implications of multi-AZ deployments (TR 7) and the complexity it introduces in resource synchronization. Balancing the need for robust security measures, such as fine-grained access control (TR 26), against user convenience and system performance was another critical tradeoff we addressed.

Furthermore, implementing stringent security controls (TR 21 & TR 26) may potentially impact system performance and complexity. For instance, the encryption of data in transit (TR 21) and fine-grained access controls (TR 26) can introduce latency and

administrative overhead. The challenge lies in enforcing robust security without compromising on the system's responsiveness and user experience.

Our commitment to these principles ensures the architecture not only supports the current operational demands but is also aligned with future scalability, reliability, and efficiency goals.

5.6 Tradeoffs revisited

1. Optimized TR6 over TR1: While scalability is crucial, cost optimization (TR6) is chosen to ensure long-term sustainability. Smart scaling uses predictive analytics to scale resources just ahead of demand spikes, striking a balance between cost and performance.

2. Optimized TR4 over TR18: The global reach of the CDN (TR4) takes precedence, ensuring all users receive a consistent experience. Content adaptation for mobile (TR18) is managed through responsive design techniques that do not heavily impact global delivery mechanisms.

3. Optimized TR20 over TR7: Edge caching (TR20) is optimized for its direct impact on user experience through reduced latency. Multi-AZ deployment (TR7) is maintained for critical components only, optimizing redundancy without excessive cost.

4. Optimized TR28 over TR11: Cost management (TR28) is essential in resource allocation, using historical traffic patterns to predictively scale resources. Load balancing (TR11) remains integral but is fine-tuned to prevent over-provisioning.

5. Optimized TR30 over TR13: Adhering to performance SLAs (TR30) is critical for customer satisfaction. Monitoring (TR13) is implemented efficiently with log aggregation and analysis to reduce overhead while still gaining insights.

6. Optimized TR19 over TR8: CDN content type optimization (TR19) is prioritized, as it directly impacts the speed and quality of content delivery. Failover strategies (TR8) are streamlined to support this without over-complicating the system architecture.

7. Optimized TR26 over TR9: Access controls (TR26) ensure data security and compliance, taking precedence over extensive real-time analytics (TR9) that could expose sensitive information.

8. Optimized TR12 over TR27: Integration ease (TR12) is prioritized to ensure quick deployment and updates. Tenant identification (TR27) systems are adapted to integrate smoothly without extensive customization.

9. Optimized TR22 over TR16: Security through SSL/TLS management (TR22) cannot be compromised. Load balancing (TR16) is adjusted to accommodate secure certificate deployment across multiple instances.

10. Optimized TR24 over TR5: The user experience benefits from real-time content compression (TR24), enhancing speed, especially for mobile users. Security protocols (TR5) are optimized to ensure minimal latency impact.

11. Optimized TR31 over TR3: Disaster recovery plans (TR31) are critical for maintaining uptime, which justifies the additional costs over stringent cost control (TR3). Backup costs are mitigated by selective backup strategies and tiered storage solutions.

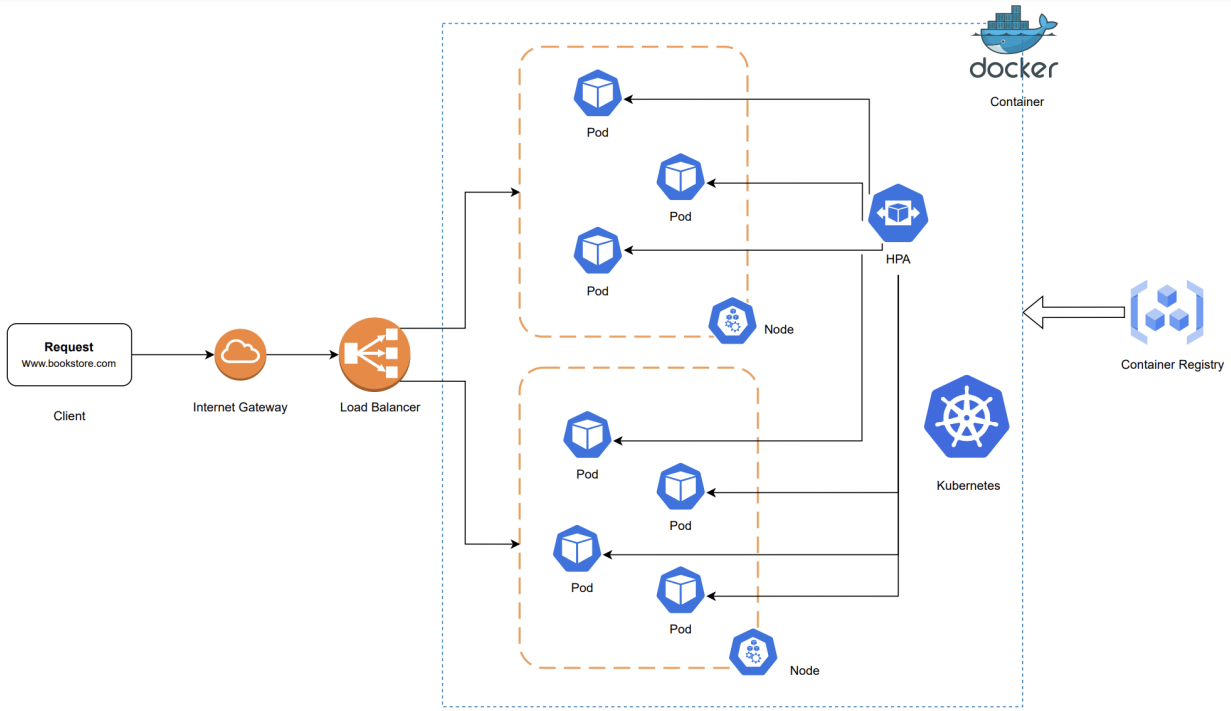
12. Optimized TR21 over TR10: Data integrity and confidentiality through HTTPS (TR21) are non-negotiable for user trust. Data visualization tools (TR10) are selected with compatibility for encrypted data processing to uphold security.

5.7 Discussion of an alternate design

Skipped

6. Kubernetes experimentation (15%)

6.1 Experiment design



Microservice Experimentation for Online Bookstore

1. Auto-Scaling Efficiency (Technical Requirement):
   A key technical requirement for the online bookstore's infrastructure is the efficient auto-scaling of resources (TR-X). The system must demonstrate the ability to automatically scale in or out, depending on the real-time load, to ensure that user requests are processed swiftly without incurring unnecessary costs during periods of low traffic. The experiment is designed to validate this auto-scaling feature, confirming that the Online Book Store within the Kubernetes environment can dynamically adjust the number of active instances (pods) based on the predefined CPU utilization threshold of 50%. This requirement directly supports the business goal of delivering a responsive and cost-effective user experience, especially during peak load times which are common in the online retail industry.
2. Experiment Details:
   a. Web Application:

- The microservice architecture for the online bookstore project is centered around a service responsible for managing customer configurations.
- This microservice, referred to as an Online Book Store, is crucial for fetching user-specific configurations from a relational database service.

b. Workflow:
   - Client requests, initiated for fetching customers' orders, are first received by the Load Balancer, which ensures that traffic is distributed optimally across the available resources.
   - This load balancer then forwards the request to the appropriate Kubernetes service (SVC) within the orchestrated cluster environment.
   - The Kubernetes SVC intelligently routes the request to one of the available pods, which are scaled and managed based on current load and performance metrics.

c. Operational Flow within Kubernetes:
   - Each pod encapsulates the Online Book Store container, which processes the incoming request by querying the RDS database to retrieve the necessary customer data.
   - After processing, the pod sends the response back through the SVC and load balancer to the client, completing the request-response cycle.

3. Necessity of Kubernetes
   a. Kubernetes addresses the scalability challenge by managing multiple instances of the Online Book Store to accommodate varying request volumes.
   b. The system utilizes horizontal pod autoscaling (HPA) to automatically adjust the number of service instances in response to the observed CPU and memory usage, effectively scaling the application without manual intervention.
4. Kubernetes Utilization
   a. The HPA monitors resource usage and scales the number of pods up or down based on predefined criteria, including a threshold for CPU utilization set at 50% for this experiment.
   b. Initially, two Kubernetes nodes, each with two pods, are employed to handle the incoming traffic.
5. Load Handling

    a. We've configured the system to operate with a minimum of one pod and a maximum of ten, allowing us to manage the service's load efficiently.
    b. If the average CPU utilization across all pods exceeds 50%, the HPA triggers the creation of additional pods to distribute the load evenly.
    c. Conversely, when CPU utilization falls below this threshold, the HPA gradually scales down the number of pods to prevent resource wastage.
6. Test Configuration
    a. We utilized the Locust tool to simulate a high volume of requests to validate the effectiveness of our Kubernetes-based autoscaling solution.
    b. The experiment successfully demonstrated the system's capacity to auto-adjust resources in response to fluctuating demand, ensuring consistent performance and resource optimization.
    c. The Docker Container Registry served as the repository for our Dockerized microservice images.

In summary, the Kubernetes architecture implemented for this online bookstore project demonstrates a resilient, scalable solution that efficiently adapts to dynamic user demands, maintaining performance and cost-effectiveness.

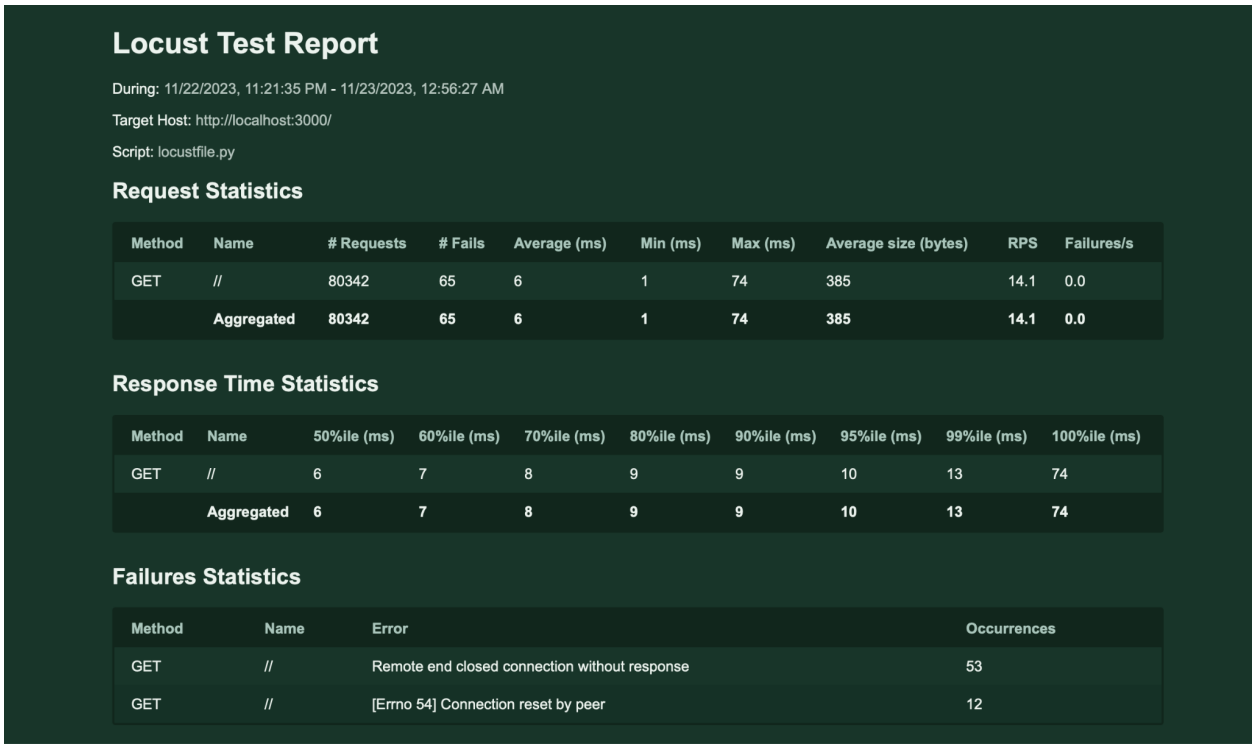
6.2 Workload generation with Locust

1. Usage of Locust tool -
2. Locust is used for load testing of an application and to check whether the system can handle concurrent requests.
3. We have used Locust to test our Online Book Store.
4. API to test - Orders()
5. Number of users (peak concurrency) - 1000
6. Spawn rate (users started/second) - 50
7. Total number of requests - 80342
8. Requests/second - 15 requests/second
9. Failure/sec - 0.2
10. Steps followed to run the locust test -
    a. Run the locustfile.py script which will start the locust server
    b. Add peak concurrency, spawn rate, number of users, and target host url
    c. Start the test

11. Our request failure rate is very less so our autoscaler handled the load and spawned new pods using our docker image stored in the registry as per the requirement.

12. When there was no request, the autoscaler decreased the replicas to the minimum replicas as expected which is shown in the next section.

13. The locust test report details:



Locust Test Report

During: 11/22/2023, 11:21:35 PM - 11/23/2023, 12:56:27 AM

Target Host: http://localhost:3000/

Script: locustfile.py

Request Statistics

Method	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	//	80342	65	6	1	74	385	14.1	0.0
	Aggregated	**80342**	**65**	**6**	**1**	**74**	**385**	**14.1**	**0.0**

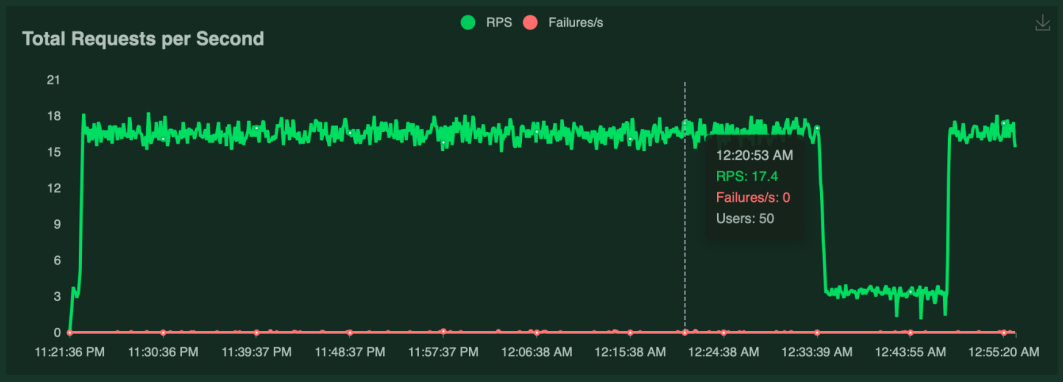
Response Time Statistics

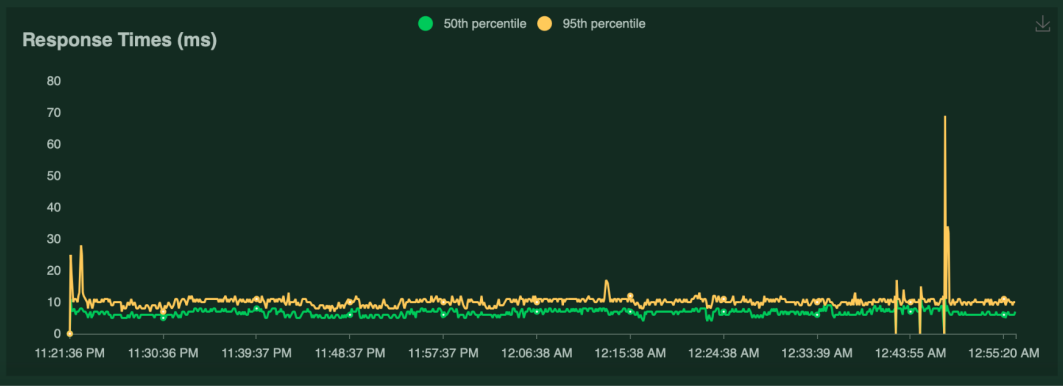
Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	//	6	7	8	9	9	10	13	74
	Aggregated	**6**	**7**	**8**	**9**	**9**	**10**	**13**	**74**

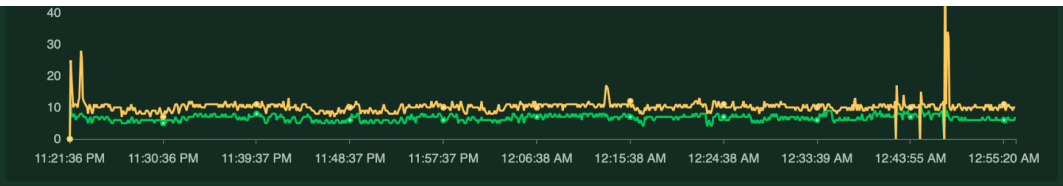
Failures Statistics

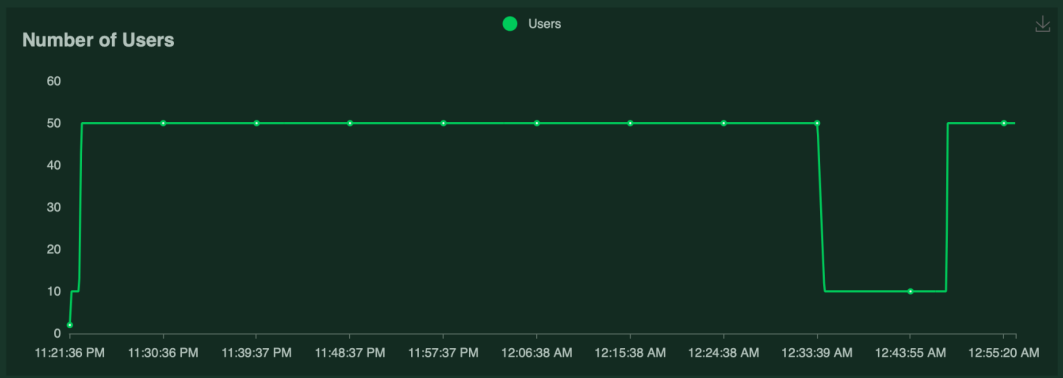
Method	Name	Error	Occurrences
GET	//	Remote end closed connection without response	53
GET	//	[Errno 54] Connection reset by peer	12

Charts









Final ratio

Ratio per User class

- 100.0% QuickstartUser
  - 100.0% hello_world

Total ratio

- 100.0% QuickstartUser
  - 100.0% hello_world

6.3 Analysis of the results

1. Initially, we have 1 Kubernetes node and 1 pod running on our service.
2. We add the autoscale by setting CPU_percent=50 i.e. mean CPU utilization=50%, minimum pod=1.
3. Name of our deployment: online-book-store-deployment
4. As our minimum replica count was 1 and maximum 10, in our deployment replica count initially was 1.
5. When we started the load testing using the Locust tool, our load generated peak concurrency and 78% request/sec, so CPU utilization of our pod increased from default.
6. In our autoscale we had set the CPU utilization to 50% so it scaled up the number of replicas in our deployment from 1 to 10.
7. As the target CPU limit increases from 70% to 82%, the number of replicas have to be autoscaled from 2 to 3.
8. When we stopped the load testing, the mean CPU utilization went to 0% so ideally we don't need 3 replicas in our deployment. Hence auto scaler scaled down the number of pods to a minimum number of pods to 1 which was the default.

```
vpatel25@vclvm177-8:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   130m   v1.28.3
vpatel25@vclvm177-8:~$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
online-book-store   1/1     1            1           123m
vpatel25@vclvm177-8:~$ kubectl get services
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes          ClusterIP   10.96.0.1       <none>        443/TCP        130m
online-book-store   NodePort    10.108.76.215   <none>        80:32000/TCP   123m
vpatel25@vclvm177-8:~$ kubectl get pods
NAME                                 READY   STATUS    RESTARTS      AGE
online-book-store-68f5b4645d-7btz6   1/1     Running   7 (57s ago)   123m
vpatel25@vclvm177-8:~$ kubectl get hpa
NAME                          REFERENCE                        TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        1          14m
vpatel25@vclvm177-8:~$ kubectl get hpa online-book-store-deployment --watch
NAME                          REFERENCE                        TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        1          15m
online-book-store-deployment   Deployment/online-book-store   70%/50%   1         10        1          15m
online-book-store-deployment   Deployment/online-book-store   70%/50%   1         10        2          16m
online-book-store-deployment   Deployment/online-book-store   82%/50%   1         10        2          16m
online-book-store-deployment   Deployment/online-book-store   68%/50%   1         10        2          17m
online-book-store-deployment   Deployment/online-book-store   68%/50%   1         10        3          18m
online-book-store-deployment   Deployment/online-book-store   52%/50%   1         10        3          18m
online-book-store-deployment   Deployment/online-book-store   24%/50%   1         10        3          19m
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        3          20m
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        3          24m
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        2          24m
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        2          25m
online-book-store-deployment   Deployment/online-book-store   0%/50%    1         10        1          25m
```

TR Validation:

Success with Auto-scaling: The experiment effectively demonstrated Kubernetes' Online Book Store's auto-scaling capabilities.

Dynamic Adjustment: In order to reach the predetermined 50% threshold, the system dynamically changed the number of pods based on the CPU consumption in real-time.

Peak Load Handling: The deployment scaled from 1 to 10 replicas during load testing, effectively handling the additional demand.

Cost Optimization: To save resource expenses during times of low traffic, the auto-scaler lowered the number of pods to the bare minimum (1).

Responsive Scalability: The auto-scaling system ensured cost-effectiveness and responsiveness by adapting well to changing workloads.

7. Ansible playbooks (0%)

Skipped

8. Demonstration (0%)

Skipped

9. Comparisons (0%)

Skipped

10. Conclusion (5%)

10.1 The Lessons Learned

1. Auto-scaling Adaptability (TR 1 & TR 2):
    - Learned the importance of dynamic resource management; auto-scaling ensured we met user demand without overspending.
    - Recognized the need for precise metric thresholds to balance resource provisioning with actual usage.

2. Redundancy for Reliability (TR 7 & TR 8):
    - Realized that multi-AZ deployment, while costly, is vital for ensuring consistent service availability.
    - Discovered the complexity of implementing seamless failover mechanisms that minimize user disruption during outages.

3. Monitoring and Operational Excellence (TR 13 & TR 14):
    - Understood that proactive monitoring is the backbone of operational excellence, allowing us to detect and respond to issues promptly.
    - Emphasized the creation of comprehensive alerts to maintain system performance and health.

4. Security Prioritization (TR 21 & TR 26):
    - Experienced the challenges of enforcing rigorous security without impeding scalability, especially during traffic spikes.
    - Observed how security measures, while essential, require careful tuning to avoid performance bottlenecks.

5. Cost-Performance Balance (TR 28):
    - Struggled with balancing performance enhancements with budgetary constraints.
    - Learned the art of optimizing resource usage without compromising system capabilities or user experience.

6. Design Trade-offs:
    - Encountered the trade-off between deploying state-of-the-art security and maintaining system flexibility.

- Grappled with the trade-off between comprehensive logging for diagnostics and the overhead it introduces.

7. Disaster Recovery Assurance (TR 31 & TR 32):
    - Recognized the importance of a robust disaster recovery plan in preserving business continuity.
    - Understood the criticality of regularly testing disaster recovery procedures to guarantee effectiveness.

8. User-Centric Focus:
    - Reaffirmed that all technical solutions must align with user needs, ensuring that features like HTTPS support (TR 21) and access control (TR 26) enhance user trust and safety.

9. Interdisciplinary Collaboration:
    - Acknowledged the value of cross-functional collaboration, where insights from various teams informed better decision-making around scalability and security.

10.2 Possible continuation of the project

None of the group members plan to take an independent study next semester, so we are not planning to continue the project.

11. References

[1] AWS Well-Architected - Build secure, efficient cloud applications - https://aws.amazon.com/architecture/well-architected/?wa-lens-whitepapers.sort-by=item.additionalFields.sortDate&wa-lens-whitepapers.sort-order=desc&wa-guidance-whitepapers.sort-by=item.additionalFields.sortDate&wa-guidance-whitepapers.sort-order=desc

[2] Comparative Analysis of Cloud Service Providers' Pricing - https://cast.ai/blog/cloud-pricing-comparison-aws-vs-azure-vs-google-cloud-platform/

[3] Amazon VPC: Design and Features - https://aws.amazon.com/vpc/features/

[4] Amazon DynamoDB: Performance and Scalability Features - https://aws.amazon.com/dynamodb/features/

[5] Application Load Balancer in AWS Ecosystem - https://aws.amazon.com/elasticloadbalancing/application-load-balancer/?nc=sn&loc=2& dn=2

[6] Security Features of Amazon S3 - https://aws.amazon.com/s3/security/?nc=sn&loc=5

[7] Encryption and Security in Amazon DynamoDB - https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/EncryptionAtRes t.html

[8] Ensuring Data Security in Amazon EC2 - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/data-protection.html#encryptio n-transit.

[9]
Amazon Well-Architected Framework: Performance Efficiency Pillar - https://docs.aws.amazon.com/pdfs/wellarchitected/latest/performance-efficiency-pillar/we llarchitected-performance-efficiency-pillar.pdf#welcome.

[10]
AWS Well-Architected Framework: Operational Excellence Pillar - https://docs.aws.amazon.com/pdfs/wellarchitected/latest/operational-excellence-pillar/w llarchitected-operational-excellence-pillar.pdf#welcome

[11]
AWS Well-Architected Framework: Reliability Pillar - https://docs.aws.amazon.com/pdfs/wellarchitected/latest/reliability-pillar/wellarchitected-r eliability-pillar.pdf#welcome

[12]
AWS Whitepaper on SaaS Solutions - https://d1.awsstatic.com/whitepapers/saas-solutions-on-aws-final.pdf

[13]
Modeling SaaS Tenant Profiles on AWS - https://aws.amazon.com/blogs/apn/modeling-saas-tenant-profiles-on-aws/ Accessed 26 November 2022.

[14]
Benefits and Configuration of Amazon EC2 Auto Scaling - https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-benefits.html#arch -AutoScalingMultiAZ

[15]
AWS Identity: Implementing Attribute-Based Access Control - https://aws.amazon.com/identity/attribute-based-access-control/

[16]
Security Pillar of the AWS Well-Architected Framework - https://docs.aws.amazon.com/pdfs/wellarchitected/latest/security-pillar/wellarchitected-s ecurity-pillar.pdf#welcome

[17]

Operational Excellence Pillar of the AWS Well-Architected Framework - https://docs.aws.amazon.com/pdfs/wellarchitected/latest/operational-excellence-pillar/we llarchitected-operational-excellence-pillar.pdf#welcome